

# Working with Claude

A field manual for building three apps with an AI pair — context, craft, parallelism, verification, and the gates that keep `main` green.

Written against your actual stack: SvelteKit 5 + Django/DRF + Supabase on Render, three sibling repos, many parallel sessions.

THE ONE IDEA

Claude’s ceiling is set by **the context you engineer**, not the prompt you type. A blunt ask against a well-instrumented repo beats a beautiful prompt against a bare one. Everything that follows is either a way of putting the right thing in front of the model *before* it starts — or a way of proving what came out.

THE LOOP

## Six moves, every task

**1**

**Frame**

Goal, constraints, and what “done” looks like. Point at the files.

**2**

**Plan**

Plan mode for anything with more than one right answer. Approve the plan, not the diff.

**3**

**Build**

Let it work. Interrupt early rather than late — a wrong direction compounds.

**4**

**Verify**

Run it. Browser, tests, screenshots. A summary is not evidence.

**5**

**Review**

`/simplify`, then `/code-review`. Fold findings into the same PR.

**6**

**Ship**

CI green, squash-merge, watch the deploy, verify the live site.

The expensive failures all come from skipping 1, 4 or 5. Step 3 is the part that feels like work and is the part you should be least involved in.

ARCHITECTURE OF MEMORY

## Four layers of context

**1 CLAUDE.md — the constitution**

Always loaded. Rules that apply to every task in that directory. Short, imperative, non-obvious only. Nested files (`/backend/`, `/frontend/`) layer on top.

**2 Skills — the playbooks**

Loaded on demand when their *description* matches what you asked. Procedure, commands, and accumulated failure modes. Where the detail belongs.

**3 Memory — the facts**

Persists across sessions: who you are, what you decided and why, what you already investigated and rejected. Cheap to write, expensive to re-derive.

**4 The session — transient and yours**

Everything read, run and said in this window. Finite, costly, and degrading as it fills. Curating it is an active job, not a background one.

CONTENTS

## What’s in here

|   |                                                                                             |      |
|---|---------------------------------------------------------------------------------------------|------|
| 2 | Context engineering — CLAUDE.md, skills, memory: what goes where, and how to write each one | p. 2 |
| 3 | Session craft — opening moves, plan mode, thinking budget, steering, context hygiene        | p. 3 |
| 4 | Working in parallel — worktrees, subagents, background work, cloud sessions                 | p. 4 |
| 5 | Verification — the browser loop, per-repo gates, and the gotchas that fake a pass           | p. 5 |
| 6 | Quality gates & shipping — review levels, PR discipline, merge races, deploys               | p. 6 |
| 7 | Automation & tooling — hooks, MCP, plugins, custom commands, headless runs                  | p. 7 |
| 8 | Beyond the terminal — models, the API in your product, and a quick-reference card           | p. 8 |

HOW TO USE IT

Read it once end to end, then treat it as a reference: pages 2 and 7 when you are *building your setup*, pages 3 to 6 when you are *in* a task, page 8 pinned next to the keyboard. Everything here is written against what is already in your repos — where it describes something you don’t have yet, that’s the suggestion, not an assumption.

## LAYER 1

# CLAUDE.md is a constitution, not a manual

It is loaded on every turn, so every line costs you on every task. A long CLAUDE.md is a diluted one — the rule you actually needed is buried among forty you didn't.

**Write only what isn't derivable**

If Claude can learn it by reading the code in ten seconds, leave it out. Keep: invariants, misconceptions it will otherwise walk into, conventions with no local example, and the consequences of getting it wrong.

**Lead with the misconception**

Your root file opens *"Architecture — read before 'fixing' the frontend"* and then says SvelteKit is *not* the server. That single paragraph prevents a whole class of confidently wrong work. Every repo has one or two of these.

**Give the rule a reason, once**

"A hand-made cover is frozen — it carries a judgement no data records." A rule with a reason generalises to cases you didn't enumerate. A bare prohibition doesn't.

**Push procedure out to skills**

The constitution says *what is always true*; a skill says *how to do this one job*. Your "Playbooks" section — a list of pointers, not procedures — is the right shape.

**Prefer a test to a bullet**

The strongest line in your backend file says a test now enforces the RLS rule and the bullet is "just context". An executable rule never rots and costs no context. Convert bullets into gates whenever you can.

**WHERE DOES THIS FACT BELONG?****THE FACT****HOME**

"Never work in the Dropbox copy"

CLAUDE.md + a PreToolUse hook

"Compile Paraglide before svelte-check"

CLAUDE.md + a PostToolUse hook

How to import and repair a book

Skill (book-import)

The Render deploy ordering trap

Skill (deploy)

"We tried normalising fixture whitespace and chose not to"

Memory

"James prefers X phrasing in PR titles"

Memory

What this function does

Nowhere — it's in the code

"Don't break the build"

Nowhere — it's noise

**IN-SESSION SHORTCUT**

Start a line with # and Claude writes the note into memory or the right CLAUDE.md for you. Use it the moment you catch yourself correcting the same thing twice. /memory opens the files for editing.

## LAYER 2

## Skills: procedures that load themselves

**The description is the trigger**

Claude sees only each skill's name and description until it opens one. Write the description in **the symptoms you'd type**: "...when a book reads wrong — bogus chapter titles, chapters split mid-sentence, a missing first letter..." fires; "book utilities" does not.

**Progressive disclosure**

Keep SKILL.md to the procedure. Push reference tables into references/ and anything deterministic into scripts/ — a script that always works beats a paragraph asking the model to be careful.

**Living playbooks**

Every one of yours ends *"append new failure modes as we find them"*, and a Stop hook nudges the review. The single highest-compounding habit in your setup: each bug you hit is paid for once.

**Anti-patterns**

- ✗ A skill that restates the code — it drifts, then lies to you.
- ✗ A description written for you, not for the trigger.
- ✗ Procedure smuggled into CLAUDE.md because it felt important.
- ✗ Two overlapping skills: Claude picks one and you never learn which.
- ✗ Memory used as a diary. Facts and decisions only, one per file.

**SCOPE**

**Repo skill** (.claude/skills/) — knowledge about *this* codebase, versioned with it. **Plugin skill** (your founder-kit) — anything that spans Ochorus, Take Root and Every Tongue. When a repo skill starts being copy-pasted to a sibling, that is the signal to promote it into the plugin.

## OPENING MOVES

## Most bad sessions were lost in the first message

A good opening prompt carries four things: the **goal**, the **constraint**, the **definition of done**, and a **pointer** to where to look. Anything you leave out, Claude will guess — competently, and sometimes not the way you meant.

## TEMPLATE

In <repo>, <goal in one sentence>. Constraints: <the invariant that must not break>. Done when: <the observable thing>. Start by reading <file> / running <command>.

## WORKED EXAMPLE

In ochorus, the Spanish edition of ‘holiness’ shows the English cover. Constraint: designed covers are frozen — don’t redraw one. Done when the es edition renders a derived ground with its own title and the digest gates still pass. Start with library/designed\_covers.py and build\_derived\_grounds.py.

### The two prompts that waste the most time

- ✗ “Fix the bug” with no reproduction. Give the input, the observed output, and the expected output — or ask it to reproduce first and report back before fixing.
- ✗ “Make it better”. Better along which axis? Name the axis and the ceiling.

## CONTEXT HYGIENE

## The window is a workspace, not a wastebasket

| MOVE                                     | WHEN                                                                                                                               |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>/clear</code>                      | New task, same repo. Almost always the right call — a fresh window with a good opening prompt beats a tired one with history.      |
| <code>/compact</code>                    | Long task you must continue. Add an instruction: <code>/compact keep the migration plan and the failing test</code> .              |
| <code>/context</code>                    | When responses get vague or slow. Shows what is eating the window — usually a giant file read or an MCP server’s tool definitions. |
| <code>/resume</code>                     | Pick up a specific earlier session by name. <code>claude -c</code> continues the most recent.                                      |
| <code>/cost</code> / <code>/usage</code> | Before you start a fan-out you can’t afford.                                                                                       |

### One job per session

The same rule as one job per branch. A session that has already imported a book, fixed a migration and argued about CSS is carrying three sets of assumptions into your fourth question. Clear it.

## PLAN BEFORE YOU BUILD

Press **Shift+Tab** to cycle into **plan mode**: Claude researches and proposes, but cannot edit. Use it whenever the task has more than one defensible shape — schema changes, a new route, anything touching the fixture or migrations. **Reviewing a plan is ten times cheaper than reviewing a diff**, and rejecting one costs you nothing.

Shift+Tab again gives **auto-accept** mode. Reserve it for work you have already scoped and can cheaply throw away — a worktree branch, never the shared checkout.

## BUY THINKING WHEN IT’S WORTH IT

The words `think`, `think hard` and `ultrathink` escalate how much reasoning Claude spends before acting. Worth it for: race conditions, migration ordering, “why is this only wrong in production”, and any design with a one-way door. Not worth it for mechanical edits — it just costs tokens and latency.

## RULE OF THUMB

Escalate thinking when you don’t know the answer either. If you already know what the fix is and just want it typed, plain mode is faster and just as correct.

## STEERING AND REPAIR

|                      |                                                                                                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Esc</b>           | Interrupt <i>now</i> . The moment you see it reading the wrong file or reaching for the wrong approach — not after the edit lands. Redirect in one sentence. |
| <b>Esc Esc</b>       | Jump back to an earlier message and take a different branch. Better than arguing forward from a bad state.                                                   |
| <code>/rewind</code> | Restore the conversation and/or the code to an earlier checkpoint. Your undo button for an agent that ran too far.                                           |
| <code>@path</code>   | Pull a specific file into context instead of hoping it searches for it.                                                                                      |
| <code>!cmd</code>    | Run a shell command yourself and put the output in context.                                                                                                  |
| <b>Ctrl+V</b>        | Paste a screenshot. For a visual bug this beats three paragraphs of description.                                                                             |

## CORRECTION BEATS PERSUASION

When output is wrong, don’t debate it — state the correct fact and the constraint it violated, then let it redo the step. “*That reverts the review state; seeds must be create-only for fields a workflow owns*” fixes it. “*No, that’s wrong*” starts a guessing game.

## ISOLATION

## One worktree, one branch, one job

You run many sessions at once across three repos. Everything on this page exists to stop them from standing on each other — and the failure mode is not a merge conflict, it's a *clean* tree that quietly lost your work.

```
# one isolated worktree per task, off the latest
main
git -C ~/dev/take-root fetch origin
git -C ~/dev/take-root worktree add \
  ~/dev/wt-<task> -b <branch> origin/main

# fresh worktrees have no gitignored files
cp ~/dev/take-root/frontend/.env frontend/.env
npm install && npx @inlang/paraglide-js compile \
  --project ./project.inlang --outdir
src/lib/paraglide

# when the branch is merged
git -C ~/dev/take-root worktree remove ~/dev/wt-
<task>
```

### The incident worth remembering

Two sessions were handed the same `claude/ochorus-dev-*` branch. The second one's book landed on the first one's open sermon PR and they shipped together under a sermon-shaped title. **Suffix the job:** `claude/ochorus-dev-<id>-<job>`. If you do land on someone else's branch, rebase onto their commits — never force-push over them — and check your own file survived with `md5` against `git show HEAD:<path>`. A rebase that replaced your file leaves the tree clean, so `git status` tells you nothing is wrong.

### Know where you are, before the first edit

Open every session in a worktree with `git status -sb`. It costs a second and catches the two expensive mistakes — you're on `main`, or you're in the shared checkout — while they are still free to fix.

### Handing a job between sessions

Hand over *artefacts*, not a conversation: a pushed branch, a PR description saying what was verified, a note in the skill. The next session then starts from the repo, not from a summary of a summary.

### WHICH MECHANISM FOR WHICH SHAPE OF WORK?

| SHAPE                                   | MECHANISM                                                                                                            |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Two features at once                    | Two <b>worktrees</b> , two terminal sessions. Never two sessions in one checkout.                                    |
| "Where is X handled?" across a big tree | <b>Subagent</b> (Explore). It reads widely and returns the conclusion, not the file dumps — your window stays clean. |
| Review along several axes               | <b>Subagents in parallel</b> , one per axis, then you synthesise.                                                    |
| A long build / test run                 | <b>Background</b> it and keep working; you're notified when it exits.                                                |
| Sequential edits to one file            | <b>Stay in the main session</b> . Fan-out here just creates conflicts.                                               |
| Poll a deploy or a queue                | <code>/loop</code> , or a scheduled agent.                                                                           |
| Work from the sofa / phone              | <b>Cloud sessions</b> at <code>claude.ai/code</code> — Every Tongue is built for exactly this.                       |

### Subagents: what they cost

A subagent gets a *fresh* context and returns only its final report. That is the whole point — it protects your window — but it also means it doesn't know what you just discussed. Brief it like a contractor: repo, goal, constraints, and the exact shape of the answer you want back.

## THROUGHPUT

## Running more than one of you

### Cloud sessions

GitHub `main` is the source of truth, no Dropbox in the loop, PRs on `claude/*` branches, CI must pass, and Render gives each PR a preview URL. The safest place to run several agents at once, because nothing shares a filesystem.

### Queue-driven work

Your translation queue is the model to copy: the admin dashboard files a GitHub issue, a session takes *one* job end-to-end, a conflict gate says which pairs collide. Machine-readable work items make parallelism safe.

### Shorthands

"**pq**" expands to the whole translation-worker playbook. Codifying your own three-word triggers in `CLAUDE.md` is free throughput — and it forces the procedure to be written down once, properly.

### THE DISCIPLINE THAT MAKES PARALLELISM PAY

`main` moves fast because many sessions land on it. Fetch and reconcile *immediately* before merging, expect fixture and migration conflicts, and never leave a branch open overnight you could land today. Parallelism converts wall-clock into merge risk; small, single-purpose PRs convert it back.

THE RULE

A summary is not evidence

The most common way an AI-assisted change goes wrong is not a bad diff — it is a *plausible report* about a diff nobody ran. Make “show me” the default.

THE BROWSER LOOP

1. Start the server from `.claude/launch.json` — never a raw `npm run dev` in a shell you then lose track of.
2. Navigate, then **read the page** (accessibility tree) rather than screenshotting: it’s faster, and it proves text and structure, which is what you usually care about.
3. Check the console and the network log for errors the UI hides.
4. Drive the interaction — click, type, submit — and re-read to confirm.
5. Resize to mobile and to dark mode if layout or theming moved.
6. **Then** screenshot, as proof for you, not as the check itself.

**Keep `launch.json` honest**

Yours points into `.claude/worktrees/...` — right for a worktree session, wrong once that worktree is gone. Prune it when you remove one, or a future session will “verify” a stale tree and pass.

**Verify what the crawler sees, not what the app renders**

Ochorus prerenders public routes for SEO. A page can look perfect in the browser and still ship an empty shell to Google. Check the *built HTML on disk* contains the real title and body — `grep` the file in `build/`, don’t trust the tab.

PER-REPO GATES

| REPO             | RUN BEFORE YOU EVEN THINK ABOUT A PR                                                                                                        |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Ochorus frontend | <code>npm run check · npm run build · npm run sync:catalogues</code> after touching <code>messages/*.json</code>                            |
| Ochorus backend  | <code>makemigrations --check</code> · tests · the fixture gates (pk rows, dangling refs, cover digests)                                     |
| Take Root        | Paraglide compile → <code>npm run check</code> → <code>npm run build</code> ; backend tests with <code>DATABASE_URL=</code> to force SQLite |
| Every Tongue     | <code>bash bootstrap.sh</code> , then backend tests + <code>npm run check &amp;&amp; npm run build</code> ; probe <code>/api/health/</code> |

**TRAPS THAT PRODUCE A FALSE GREEN**

- **Prerender needs the API up.** A build that “passed” with the backend down proves nothing about SEO pages.
- **Paraglide output is generated, not committed.** A fresh worktree fails `check` until you compile it.
- **backend/.env points at production Supabase.** Never run the suite without blanking `DATABASE_URL`.
- **A stale vite preview** serves the previous build happily. Rebuild, then restart it.
- **Symlinked node\_modules** in a worktree can silently kill hydration.

PROMPTING FOR PROOF

Ask for the evidence, not the verdict

**INSTEAD OF “DOES IT WORK?”**

Run it and show me: the failing case before the fix, the same case after, and the exact command you ran. If you can’t run it, say so — don’t infer.

**FOR A UI CHANGE**

Start ochorus-frontend, open `/books/<slug>`, screenshot at desktop and mobile, paste any console errors, then say what a reviewer would object to.

**Two questions worth asking every time**

“What did you *not* verify?” — surfaces the untested branch before your users do.

“What would make this wrong?” — invites the model to argue against its own change, which it does well and rarely volunteers.

**Make failures loud in the repo**

Your RLS test, the fixture gates and the cover digests all turn “remember to...” into “CI fails”. Every time you catch Claude making the same mistake twice, ask whether it should have been a test instead of a sentence.

THE LADDER

Match the proof to the claim

| THE CLAIM        | WHAT ACTUALLY SETTLES IT                                          | WHAT DOESN'T                           |
|------------------|-------------------------------------------------------------------|----------------------------------------|
| “It compiles”    | A clean <code>check</code> and <code>build</code> , output pasted | “the types look right”                 |
| “It works”       | The interaction driven in a browser, console clean                | A screenshot of the page loading       |
| “It’s indexable” | The title and body found in the built HTML on disk                | The tab title in a dev server          |
| “It’s live”      | A new entry-chunk hash on <code>ochorus.com</code>                | A green deploy in the Render dashboard |

## BEFORE THE PR

## Two passes, in this order

**/simplify** — first

Quality only: reuse, simplification, efficiency, altitude. It does not hunt for bugs. Run it first because it *changes the code*, and reviewing simplified code is cheaper than reviewing the sprawl that came out of a build session.

**/code-review high** — second

Correctness. Levels are a real dial: **low/medium** gives few, high-confidence findings; **high-max** broadens coverage and will include uncertain ones. Use **high** for anything logic-bearing — migrations, auth, the seed/upsert paths, anything touching money or content integrity.

**--fix** applies findings to the working tree; **--comment** posts them inline on the PR. **/code-review ultra** runs a deep multi-agent review in the cloud — save it for a change you'd lose sleep over.

**/security-review**

Worth a run on anything that adds an endpoint, touches **permission\_classes**, changes sanitisation, or moves a key. Your DRF default is **AllowAny** — a forgotten **IsAdminEmail** ships an open, DB-mutating endpoint, and that is a review-shaped bug, not a test-shaped one.

## FOLD, DON'T FOLLOW UP

Findings go into **the same PR**. A “cleanup PR to follow” is a PR that doesn't happen. Your **gh pr create** hook exists precisely to catch you here.

## PR DISCIPLINE

- ✓ **One feature per PR**. Not because it's tidy — because a mixed PR can't be reverted.
- ✓ **Explicit paths**. Stage the files you meant; **git add -A** in a worktree with generated output is how junk lands.
- ✓ **Title says what changed for a user**, body says why and what you verified. Your history reads well — keep that.
- ✓ **Check CI before merging** (**gh pr checks <n>**). Render auto-deploys on merge; a red merge is a live outage.
- ✓ **Squash-merge**, then delete the branch and remove the worktree.

## THE MERGE RACE

**main** moves under you. Fetch and rebase *immediately* before merging, and expect the two conflict classes that actually bite:

## Fixtures

One file per work makes most collisions impossible — that design is doing real work for you. Conflicts that remain are usually **authors.json** appends.

## Migrations

Two branches both numbered **0092**. Resolve by renumbering and re-checking **makemigrations --check**, not by deleting one.

## AFTER THE MERGE

1. Watch *both* Render services — the API runs **manage.py release** (migrate → seeds → backfills) as a pre-deploy step, the static site runs its own build.
2. Confirm the deploy actually shipped: compare the entry-chunk hash, not the page's appearance.
3. Verify on the URL readers use — **ochorus.com**, not just the Render hostname.
4. If content changed in the DB, remember the static pages need a redeploy to catch up.

## CHECKLIST

## The last ten minutes before merge

## Correctness

- ✓ Gates run *after* the review fixes, not before.
- ✓ Migrations: **makemigrations --check** is clean and the numbering doesn't collide.
- ✓ New endpoint? **permission\_classes** set explicitly.
- ✓ New table? RLS on, and the test agrees.

## Hygiene

- ✓ Only the files you meant are staged.
- ✓ No generated output committed (Paraglide, build artefacts).
- ✓ No secrets, no **.env**, no prod connection string.
- ✓ Debug logging and scratch files removed.

## Blast radius

- ✓ Could this revert human work on the next deploy? (See below.)
- ✓ Does it touch a frozen artefact — a designed cover, an approved translation?
- ✓ If it's wrong in production, how do you notice, and how fast can you revert?

## SEEDS RE-RUN ON EVERY DEPLOY — THE RULE THAT FOLLOWS FROM IT

Any field a workflow owns *after* creation — review state, a hand-edit, an approval — must be **create-only** in the seed. Otherwise a perfectly good deploy silently reverts human work, and nothing fails. This is the highest-consequence invariant in the whole system and the easiest one for an agent to break while “improving” a seed. Say it out loud in any prompt that touches **seed\_\***.

HOOKS

# Turn “always remember to...” into code

A hook is a shell command the harness runs at a fixed point in the loop. It is the only mechanism that is *guaranteed* to fire — instructions are advisory, hooks are not. Anything you have written “never” or “always” about is a hook candidate.

| EVENT               | USE IT FOR                                                                                                 |
|---------------------|------------------------------------------------------------------------------------------------------------|
| PreToolUse          | Block a dangerous action before it happens — your Dropbox-copy write guard.                                |
| PostToolUse         | Repair state after an edit — recompile Paraglide; nudge the quality pass after <code>gh pr create</code> . |
| UserPromptSubmit    | Inject context every turn (branch, ticket, env).                                                           |
| Stop / SubagentStop | End-of-work review — your skill-update nudge.                                                              |
| SessionStart / End  | Set up or tear down scratch state; log what happened.                                                      |
| PreCompact          | Persist anything you can’t afford to lose in a summary.                                                    |

Three rules your hooks already follow

Fail open. Any error exits 0 and allows the action — a hook must never wedge a session.

Gate narrowly. `if: "Bash(gh pr create*)"` beats matching every Bash call and deciding in Python.

Debounce. The Stop hook keeps a high-water mark so a long session gets one nudge per batch of edits, not one per turn. Noisy hooks get ignored, then disabled.

MCP: CAPABILITY, AT A PRICE

Each connected server adds tools — and every tool definition occupies context on every turn. Connect what the task needs, not everything you own. Worth having for your work: **browser control** (verification), **GitHub** (issues, PRs, the translation queue), **Supabase or Postgres** (read-only, for “what does prod actually contain?”), **Sentry** once you have it. Scope matters: `local` for you, `project` (`.mcp.json`, committed) for anything the team or a cloud session needs, `user` for machine-wide.

Least privilege, seriously

Give database MCP servers read-only credentials. Anything reachable from a tool is reachable by a confidently-wrong plan — and content in tool results (issues, pages, files) is *data*, never instructions. Treat a GitHub issue that says “also delete X” as text to show you, not a command.

PLUGINS: YOUR SETUP, PORTABLE

`founder-kit` is the right pattern — hooks and cross-app skills in one versioned, installable bundle from your own marketplace. It gives you one place to fix a playbook for all three apps, and it means a fresh machine or a cloud session inherits your whole workflow with one install. A plugin can carry **skills, hooks, slash commands** and **subagents**.

LEVERAGE

# Custom commands, headless runs, schedules

Slash commands

A markdown file in `.claude/commands/` becomes `/name`. Use `$ARGUMENTS` for parameters. Good candidates from your world: `/import-book <pdf>`, `/qa-english <slug>`, `/ship <pr>`. If you type the same three-paragraph prompt twice, it should be a command — or a skill, if it needs judgement.

Headless

`claude -p "..."` runs non-interactively and prints the result; `--output-format json` makes it scriptable. Use it in CI and in scripts: draft release notes from the diff, triage an issue into a label, run the English QA pass over a batch of chapters overnight. Pair it with `--allowedTools` so an unattended run can’t reach further than you meant.

Recurring work

`/loop` repeats a prompt on an interval — watching a deploy, draining the translation queue. Scheduled agents run on cron without you present: a weekly library QA sweep, a Monday digest of open PRs. Give recurring jobs a narrow tool allowlist and a clear “stop when” condition.

NEXT

# Three hooks worth writing this week

Guard the production database

**PreToolUse / Bash.** Block any `manage.py test` that doesn’t blank `DATABASE_URL`. Take Root’s `.env` points at real Supabase — today that rule is a sentence, one distracted session from being expensive.

Keep the catalogues in sync

**PostToolUse / Edit.** When `messages/*.json` changes, run `npm run sync:catalogues` automatically — the same shape as your Paraglide hook. A test already catches it; a hook means you never see the failure.

Announce the branch

**SessionStart.** Print the repo, branch, worktree path, and how far behind `origin/main` it is. Cheap insurance against the branch-collision failure on page 4, and it puts the answer in context before the first question.

FEWER INTERRUPTIONS

Permission prompts train you to click “yes” without reading, which is the real danger. Allowlist the read-only things you approve fifty times a day — `git status`, `git diff`, `ls`, `npm run check` — so that when a prompt does appear it means something. Keep writes, pushes and deploys on the list.

PAGE 7 OF 8

## MODEL &amp; API

## Claude inside the product, not just beside it

## CHOOSING A MODEL

|        |                                                                                                                                       |
|--------|---------------------------------------------------------------------------------------------------------------------------------------|
| Opus   | Hard reasoning, architecture, gnarly debugging, long agentic runs, and any translation or editorial pass where nuance is the product. |
| Sonnet | The everyday driver — most feature work, refactors, review passes. Fast and strong.                                                   |
| Haiku  | High-volume, low-judgement: classification, extraction, formatting a thousand chapters. Cheap enough to run over the whole library.   |

Switch mid-session with `/model`. A useful pattern: plan on Opus, execute on Sonnet, batch-process on Haiku.

## Patterns your pipelines should use

**Prompt caching** — a long, stable prefix (style guide, glossary, book context) cached across every chapter is a large cost and latency win on exactly your workload.

**Batch processing** for anything not interactive — a whole book's translation at a discount.

**Structured output** — force a schema instead of parsing prose. Fewer failure modes, and validation happens before your code sees it.

**Evals before scale** — twenty hand-checked chapters tell you more about a prompt change than a thousand unchecked ones.

## You already have the most important guardrail

AI translations ship as `ai_unreviewed`, wear an “awaiting native review” badge, and only a human approval upgrades them. Never auto-approve; never present an unreviewed translation as an original. Any new AI feature should copy that shape: **generate freely, publish only through a human gate**.

## THE SHORT VERSION

## Twelve habits

- 1. Frame before you ask.** Goal, constraint, done, pointer.
- 2. Plan mode for anything with two right answers.** Reviewing a plan is ten times cheaper than reviewing a diff.
- 3. One job per session, per branch, per worktree, per PR.**
- 4. Interrupt early.** The cost of a wrong direction compounds every turn.
- 5. Never accept a report as proof.** Ask for the command and its output.
- 6. Prefer a test to a sentence.** A rule in CI never rots; a rule in prose does.
- 7. Write down every failure the moment it happens** — into the skill, not into your head.
- 8. Curate the context window** the way you'd curate a workbench.
- 9. `/simplify` then `/code-review`, folded into the same PR.**
- 10. Fetch immediately before merging.** `main` moved while you were reading this.
- 11. Generate freely; publish through a human gate.**
- 12. When you correct the same thing twice, that's a bug in your setup** — not in the model. Fix the setup.

## HONEST ASSESSMENT

## Where you're strong, and where to push

## Already ahead of most

Short, opinionated CLAUDE.md files that lead with the misconception. Living-playbook skills with real failure modes in them. A plugin that carries hooks and playbooks across three repos. Invariants enforced by tests rather than by prose. A human approval gate on AI output. Worktree isolation with a written record of the incident that taught you why.

## BEYOND THE TERMINAL

**Artifacts** for anything with an audience — a strategy note, a spec, a report someone else will read. **Projects** for a standing context you return to (the style guide, the language roadmap). **Claude in the browser** for the parts of shipping that aren't code: dashboards, Render, Supabase config.

**Voice / mobile** for thinking a problem through before you sit down to it — a good plan written on a walk is worth an hour at the keyboard.

## QUICK REFERENCE

| TYPE THIS                       | TO GET                                            |
|---------------------------------|---------------------------------------------------|
| <b>Shift+Tab</b>                | Cycle plan mode / auto-accept                     |
| <b>Esc · Esc Esc</b>            | Interrupt · jump back and re-branch               |
| <b># · @ · !</b>                | Remember · reference a file · run a shell command |
| <b>/clear · /compact</b>        | Fresh window · summarise and continue             |
| <b>/context · /cost</b>         | What's filling the window · what it cost          |
| <b>/rewind · /resume</b>        | Undo to a checkpoint · reopen a session           |
| <b>/simplify · /code-review</b> | The two pre-PR passes                             |
| <b>/agents · /mcp · /hooks</b>  | Manage subagents · servers · hooks                |
| <b>claude -c · claude -p</b>    | Continue last session · headless run              |
| <b>/init · /memory</b>          | Bootstrap CLAUDE.md · edit memory                 |

## The next three moves

- Turn the remaining “never” sentences into hooks or tests — start with the production-database guard.
- Make verification the default rather than a request: put the browser loop and the per-repo gates into a `verify` skill that every ship playbook calls.
- Promote whatever you've copy-pasted between Ochorus and Take Root into `founder-kit`, so Every Tongue inherits it on day one instead of re-earning it.